Testen mit JUnit (JUnit-Basics)

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Software-Fehler und ihre Folgen

1982: Absturz eines F117 Kampfjets: in der Softwaresteuerung Höhen- und Seitenruder vertauscht
http://de.wikipedia.org/wiki/Programmfehler

2005: TOKIOTER BÖRSE: Chef tritt ab wg. Softwarefehler (mehrere 100 Millionen Dollar Verlust)
http://www.manager-magazin.de/unternehmen/karriere/0,2828,druck-391434,00.html

2006: VW ruft 3500 Passat-Modelle zurück. Ein Softwarefehler kann zum Absterben des Motors führen.
http://auto.t-online.de/volkswagen-rueckruf-fuer-den-passat-wegen-softwarefehler/id_12821896/index

2007: Defektes Computersystem für den Tod von zehn Soldaten verantwortlich
http://www.heise.de/newsticker/meldung/Defektes-Computersystem-fuer-den-Tod-von-zehn-Soldaten-verantwortlich-186999.html?view=print

2008: Volvo-Rückruf: XC90 mit Software-Problem (Zündung vs. Klimaanlage)
http://www.auto-motor-und-sport.de/news/volvo-rueckruf-xc90-mit-software-problem-711983.html

2010: Softwarefehler in EC-Karten-Sicherheitschip: Kunden bekommen kein Geld am Automaten
http://www.tagesspiegel.de/wirtschaft/ec-karten-panne-zum-jahreswechsel/1658102.html

2010: Rückrufe kosten Toyota mehr als eine Milliarde Euro (Softwareprobleme)
http://www.spiegel.de/wirtschaft/unternehmen/0,1518,druck-675874,00.html

2010: Ford: Rückrufe wg. Softwareproblemen im Bremssystem
http://www.automobil-produktion.de/2010/02/ruckruf-jetzt-auch-ford/

2011: Honda startet umfangreiche Rückrufaktion, weltweit etwa 2,5 Millionen Autos betroffen (Softwareprobleme können zu Schäden am Getriebe führen)
http://www.handelsblatt.com/unternehmen/industrie/softwareprobleme-honda-startet-umfangreiche-rueckrufaktion/v_detail_tab_print/4470752.html

2012: Report "Softwareentwicklung 2012" (Accso): "Softwarefehler kosten die deutsche Volkswirtschaft Milliarden"
http://www.elektronikpraxis.vogel.de/index.cfm?pid=5416&pk=361330&print=true&printtype=article

Ursachen für Fehler?

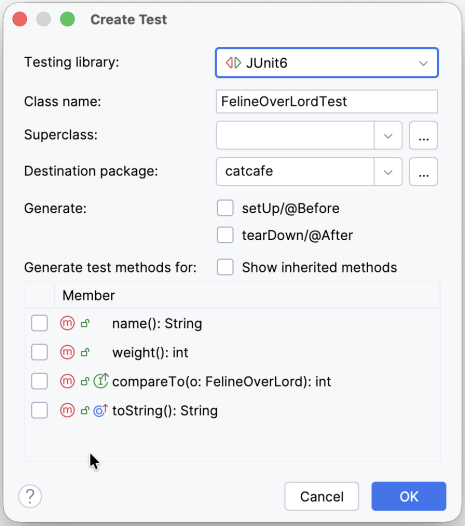
Zentrale Begriffe: Was wann testen?

- **Modultest / Unit-Test**
  - Testen einer Klasse und ihrer Methoden
  - Test auf gewünschtes inneres Verhalten (Parameter, Schleifen, ...)
- **Integrationstest**
  - Test des korrekten Zusammenspiels mehrerer Komponenten
  - Fokus auf Schnittstellen, Datenaustausch, Zusammenspiel
- **Systemtest / End-to-End-Test (E2E)**
  - Test des kompletten Systems unter produktiven Bedingungen
  - Fokus: Gesamter Geschäftsprozess, UI, Use Cases
  - Funktionale und nichtfunktionale Anforderungen testen
- **Regressionstest**
  - Änderungen dürfen bestehende Funktionalität nicht brechen
  - Fokus: Wiederholung von bestehenden Tests nach Code-Änderungen

=> Verweis auf Wahlfach "Softwarequalität"

Anlegen und Organisation der Tests mit JUnit

- Anlegen neuer Tests: Klasse auswählen, Kontextmenü `Generate > Test`
- **Best Practice**: Spiegeln der Paket-Hierarchie
  - Toplevel-Ordner `src/test/java/` (statt `src/main/java/`)
  - Package-Strukturen spiegeln
  - Testklassen mit Suffix "`Test`"



Definition von Testmethoden

Annotation `@Test` vor Testmethode schreiben

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class DemoResults {
    @Test
    void passes_when_assertion_is_true() {
        assertEquals(0, 1 - 1);
    }
}
```

Ergebnis prüfen

Klasse `org.junit.jupiter.api.Assertions` enthält diverse **statische** Methoden zum Prüfen:

```
// Argument muss true bzw. false sein
static void assertTrue(boolean condition);
static void assertFalse(boolean condition);

// Gleichheit im Sinne von equals()
static void assertEquals(Object expected, Object actual);
static void assertEquals(long expected, long actual);

// Test sofort fehlschlagen lassen
static <V> V fail(String message);

...
```

Mögliche Testausgänge bei JUnit: rot/grün/ignoriert

```
class DemoResults {
    @Test
    void passes_when_assertion_is_true() {
        assertEquals( expected: 0,  actual: 1 - 1); // assert() ist ok → test ist "grün"
    }

    @Test
    void fails_when_assertion_is_false() {
        assertEquals( expected: 0,  actual: 1); // assert() ist NOK → test ist "rot"
    }

    @Test
    void fails_when_unexpected_exception_is_thrown() {
        throw new RuntimeException(); // nicht gefangene exception → test ist "rot"
    }

    @Test
    void passes_when_no_assertion_fails_and_no_exception_occurs() {
        int x = 42; // kein fehlschlagendes assert(), keine exception → test ist "grün"
    }

    @Test
    void is_skipped_when_assumption_is_not_met() {
        assumeTrue( assumption: 1 == 0); // bricht ab und ignoriert den test

        assertEquals( expected: 0,  actual: 1);
    }
```

```
2 tests failed, 2 passed, 1 ignored   5 tests total, 22 ms

Expected :0
Actual   :1
<Click to see difference>

org.opentest4j.AssertionFailedError Create breakpoint : expected: <0> but was: <1> <6 internal lines>
    at demo.DemoResults.fails_when_assertion_is_false(DemoResults.java:17)
```

Anmerkung zu Asserts pro Testmethode

BDD: “Given - When - Then”-Mantra

```java
// Before BDD
class StudiTest {
    @Test
    void testAddToCredits() {
        Studi s = new Studi();
        int cps = 2;
        s.addToCredits(cps);
        assertEquals(cps, s.getCredits());
    }
}
```

```java
// With BDD: given-when-then
class StudiTest {
    @Test
    void accumulates_credits_within_upper_limit() {
        // given
        var studi = new Studi();
        var startCredits = 2;

        // when
        studi.addToCredits(startCredits);
        var endCredits = studi.getCredits();

        // then
        assertEquals(startCredits, endCredits);
    }
}
```

Test-Fixtures: Testübergreifende Konfiguration

```java
private Studi x;

@BeforeEach
void setUp() { x = new Studi(); }

@Test
void toString_formats_name_and_credits() {
    // Studi x = new Studi();
    assertEquals("Heinz (15cps)", x.toString());
}
```

- Vor/nach *jedem* einzelnen Test:
  - `@BeforeEach` : wird **vor jeder** Testmethode aufgerufen
  - `@AfterEach` : wird **nach jeder** Testmethode aufgerufen
- Einmalig (Klassenmethoden):
  - `@BeforeAll` : wird **einmalig** vor allen Tests aufgerufen (`static`!)
  - `@AfterAll` : wird **einmalig** nach allen Tests aufgerufen (`static`!)

Test von Exceptions

```java
@Test
void divisionByZero_throwsArithmeticException() {
    try {
        int i = 0 / 0;
        fail("keine ArithmeticException ausgeloest");
    } catch (ArithmeticException aex) {
        assertNotNull(aex.getMessage());
    } catch (Exception e) {
        fail("falsche Exception geworfen");
    }
}
```

Test von Exceptions

```java
@Test
void divisionByZero_throwsArithmeticException() {
    try {
        int i = 0 / 0;
        fail("keine ArithmeticException ausgeloest");
    } catch (ArithmeticException aex) {
        assertNotNull(aex.getMessage());
    } catch (Exception e) {
        fail("falsche Exception geworfen");
    }
}
```

```java
@Test
void divisionByZero_throwsArithmeticException() {
    assertThrows(java.lang.ArithmeticException.class, () -> { int i = 0 / 0; });
}
```

Parametrisierte Tests

```
class Sum {
    public static int sum(int i, int j) {
        return i + j;
    }
}

class SumTest {
    @Test
    void sum_of_one_and_one_is_two() {
        assertEquals(2, Sum.sum(1, 1));
    }
    // und mit (2,2, 4), (2,2, 5), ...????
}
```

```java
@ParameterizedTest
@ValueSource(strings = {"wuppie", "fluppie", "foo"})
void testWuppie(String candidate) {
    assertTrue(candidate.equals("wuppie"));
}
```

```java
@ParameterizedTest
@CsvSource({
        // s1,  s2,  result
        "0,     0,      0",
        "10,    0,      10",
        "0,     11,     11",
        "-2,    10,     8"
        })
void sum_adds_two_integers(int s1, int s2, int erg) {
    assertEquals(erg, Sum.sum(s1, s2));
}
```

Wrap-Up

JUnit als Framework für (Unit-) Tests

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbedingung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.